

Data Migration and Resilience



Building Resilience and High Availability in the Cloud



Engage and Think



You manage a mission-critical cloud app built for high availability across regions. Suddenly, one region shows latency spikes and errors. Auto-scaling and health checks are active; however, issues persist. Users are affected.

What is the first action you should take to maintain availability and reduce disruption during this regional incident?

Learning Objectives

By the end of this lesson, you will be able to:

- Explain the key concepts of cloud resilience, including fault tolerance, high availability, and automated recovery
- Differentiate between high availability and disaster recovery, and define recovery objectives such as RTO and RPO
- Design resilient database architectures using RDS Multi-AZ for failover and Read Replicas for scaling and disaster recovery
- Develop effective backup and restore strategies with AWS Backup, EBS Snapshots, and S3 Versioning
- Implement disaster recovery patterns and services, including Route 53, Elastic Load Balancer, and Auto Scaling, to minimize downtime and maintain availability

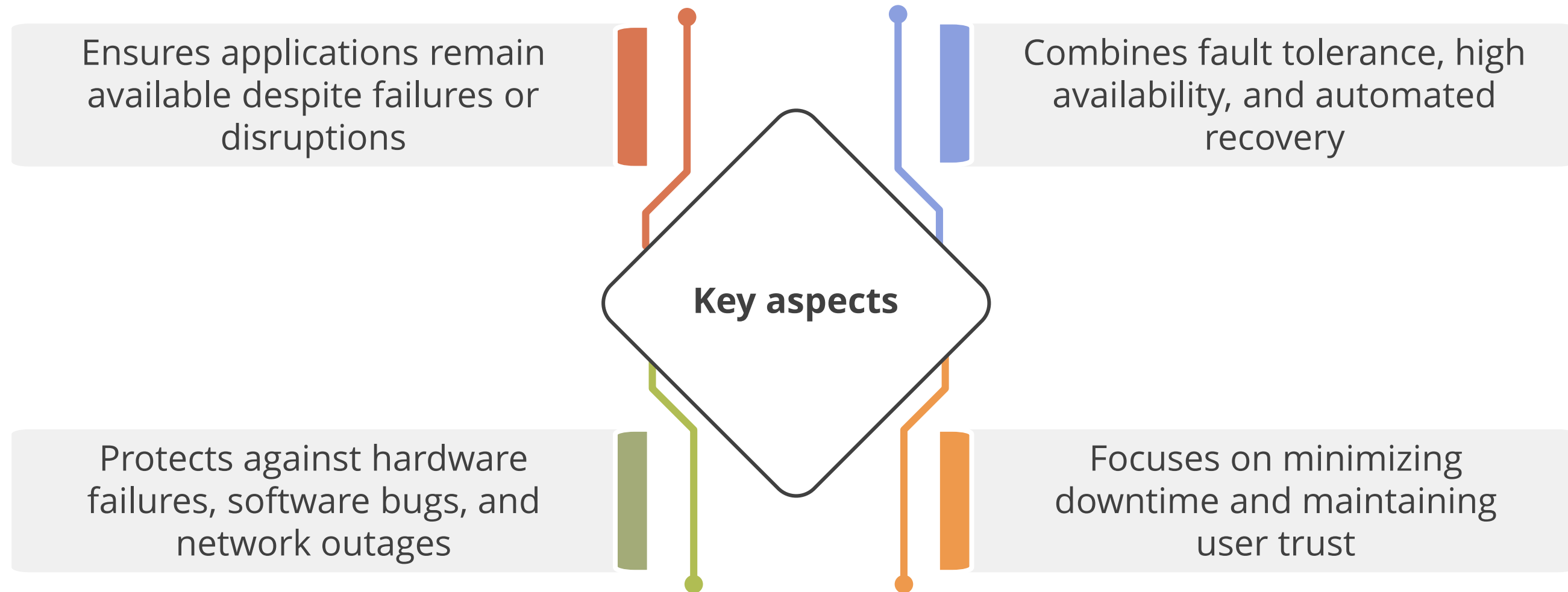




Understanding Resilience in Cloud Computing

Resilience: Overview

It is the ability of a system to recover quickly and continue operating during disruptions, and it is achieved through the following aspects:



Example: An e-commerce platform that automatically reroutes traffic to a healthy region when one data center fails.

Resilience: Importance

Building resilience in cloud systems helps address critical operational and business challenges.
The following are four key benefits that resilient cloud architectures deliver:

Ensure business continuity

Protects revenue, reputation, and customer trust by reducing downtime

Handle dynamic workloads

Manages unpredictable spikes in demand through automatic scaling

Support global operations

Delivers consistent service across multiple regions worldwide

Reduce operational risks

Prevents outages from hardware failures, software bugs, or cyberattacks

Resilience: Core Principles

Fault tolerance and high availability

Keeps systems running during hardware or software failures

Scalability and elasticity

Dynamically adjusts resources to meet demand without service interruption

Multi-AZ and multi-region deployments

Reduces risk by distributing infrastructure across locations

Disaster recovery planning

Ensures quick recovery and acceptable data loss during outages

Resilience: Core Principles

Decoupled and loosely coupled architectures

Prevents failures from spreading by isolating components

Monitoring, alerts, and auto-healing

Identifies issues early and triggers automatic recovery processes

Infrastructure as code (IaC)

Enables consistent recovery and scalable, automated deployments

Cloud-native best practices

Provides design guidelines to build resilient systems systematically

Use Case

Capital One - Reliability with AWS Well-Architected Principles



Problem

- Capital One operated legacy data centers that limited scalability, agility, and innovation.
- The infrastructure failed to meet the growing demand for real-time, personalized customer experiences.
- IT teams experienced long provisioning times, often taking months to deliver infrastructure and environments.
- On-premises systems created barriers to disaster recovery, resilience, and cloud-native transformation.

Use Case

Capital One - Reliability with AWS Well-Architected Principles



Solution

- Capital One migrated fully to AWS and adopted a cloud-first model to replace on-premises data centers.
- The organization built a microservices-based architecture to enable modular and scalable system design.
- Infrastructure provisioning was automated to deliver resources instantly and improve cost efficiency.
- Active-active, cross-region architecture was implemented to achieve high availability and fault tolerance.

Use Case

Capital One - Reliability with AWS Well-Architected Principles



Outcome

- System uptime improved to 99.99 percent across all critical applications.
- Recovery times were reduced from several hours to just a few minutes.
- Regulatory compliance was enhanced while operational overhead was significantly lowered.

Quick Check

A fintech company experiences a traffic spike during a national sale. Soon after, a server fails, causing service disruption. The system was not designed to handle sudden loads or isolate failures. Which resilience principles could have prevented this issue?

- A. Use of a single availability zone and vertical scaling only
- B. Manual provisioning and centralized service dependencies
- C. Multi-region deployment with load balancing and auto-scaling
- D. Frequent backups and monthly disaster recovery drills only





Designing for High Availability and Disaster Recovery

High Availability: Overview

It is a key design principle that ensures cloud services remain operational, even in the event of failures.

It uses redundant resources and automatic failover to reduce service downtime.

It is achieved by deploying systems across multiple zones and by using load-balancing techniques.

Example: An online retail platform uses AWS Auto Scaling and Elastic Load Balancer to keep its application available during peak traffic or instance failure.

Disaster Recovery: Overview

It is the process of restoring systems and data after a serious disruption or failure.

It includes regular backups, snapshot replication, and region failover.

It defines how quickly systems must recover (RTO) and how much data loss is acceptable (RPO).

Example: A healthcare company stores encrypted backups in a separate AWS region and uses pilot light architecture to quickly restore services after a ransomware attack.

AWS RDS Multi-AZ vs. Read Replicas

To implement high availability and disaster recovery for databases, AWS RDS offers two key features:

Multi-AZ

It provides automatic failover to a standby instance in another availability zone to ensure high availability.

Read Replica

It supports horizontal scaling and disaster recovery by asynchronously copying data to separate instances.

Example:

A financial app uses Multi-AZ for availability and Read Replicas for reporting and recovery.

Backup and Restore Strategies in AWS

AWS provides built-in tools to back up data and recover quickly from accidental deletions, failures, or disasters. The following are the tools:



Example:

A software company uses AWS Backup to schedule daily backups of databases, EBS Snapshots to protect EC2 volumes, and S3 Versioning to recover accidentally deleted files.

Automating Protection with AWS Backup

It provides centralized, automated backup management across supported AWS services.

The following are the features:

- Supports scheduled, policy-based backups for services like RDS, DynamoDB, EFS, and EC2
- Enables backup retention and lifecycle management for compliance and cost control
- Offers cross-region backup for geographic resilience and regulatory requirements

Protecting Volumes with EBS Snapshots

It provides volume-level, point-in-time backups for EC2 instances. The following are the features:

- Creates incremental, space-efficient backups stored in Amazon S3
- Restores EC2 volumes quickly in case of data corruption or application failure
- Integrates with AWS Backup and lifecycle policies for automation and retention

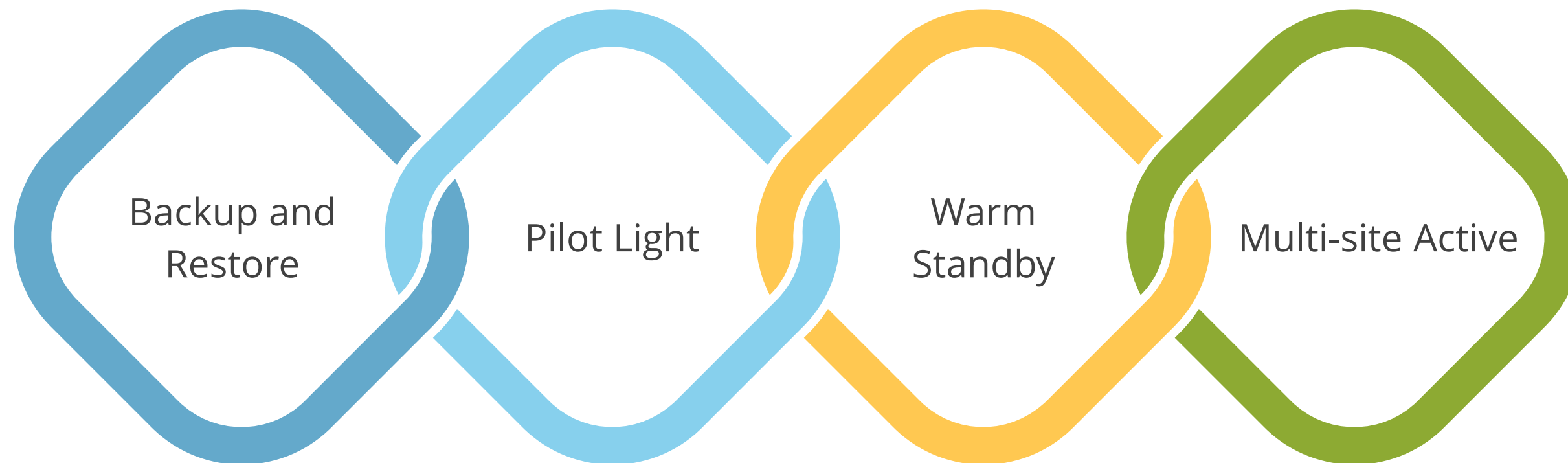
Recovering Data with S3 Versioning

It stores all object versions, helping protect against data loss from overwrites or deletions. The following are the key features:

- Preserves every version of an object, even after deletion or update
- Enables rollback to earlier versions in case of accidental change
- Works with lifecycle rules to manage storage costs from version sprawl

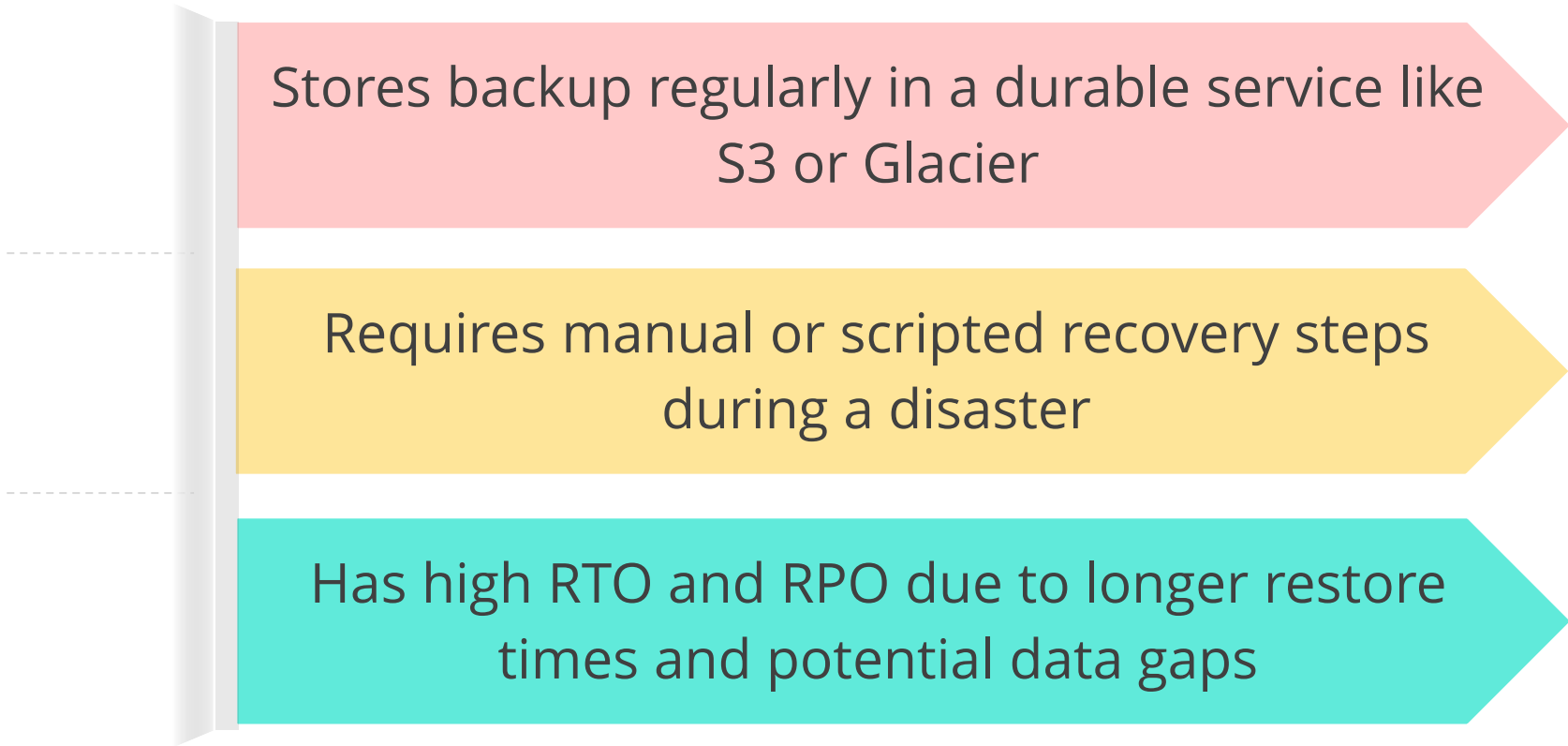
Disaster Recovery Patterns: Overview

It defines structured approaches to restore systems and data during a failure or large-scale disruption.
The following four DR patterns are commonly used in cloud architecture:



Backup and Restore Pattern

It is the simplest disaster recovery strategy, focused on storing data and restoring it only after a failure occurs.



The diagram consists of three horizontal chevron-shaped boxes stacked vertically, each containing a characteristic of the Backup and Restore Pattern. The top box is pink, the middle is yellow, and the bottom is teal. A vertical grey bar is on the left side of the boxes, and dashed horizontal lines extend from the bar to the start of each box.

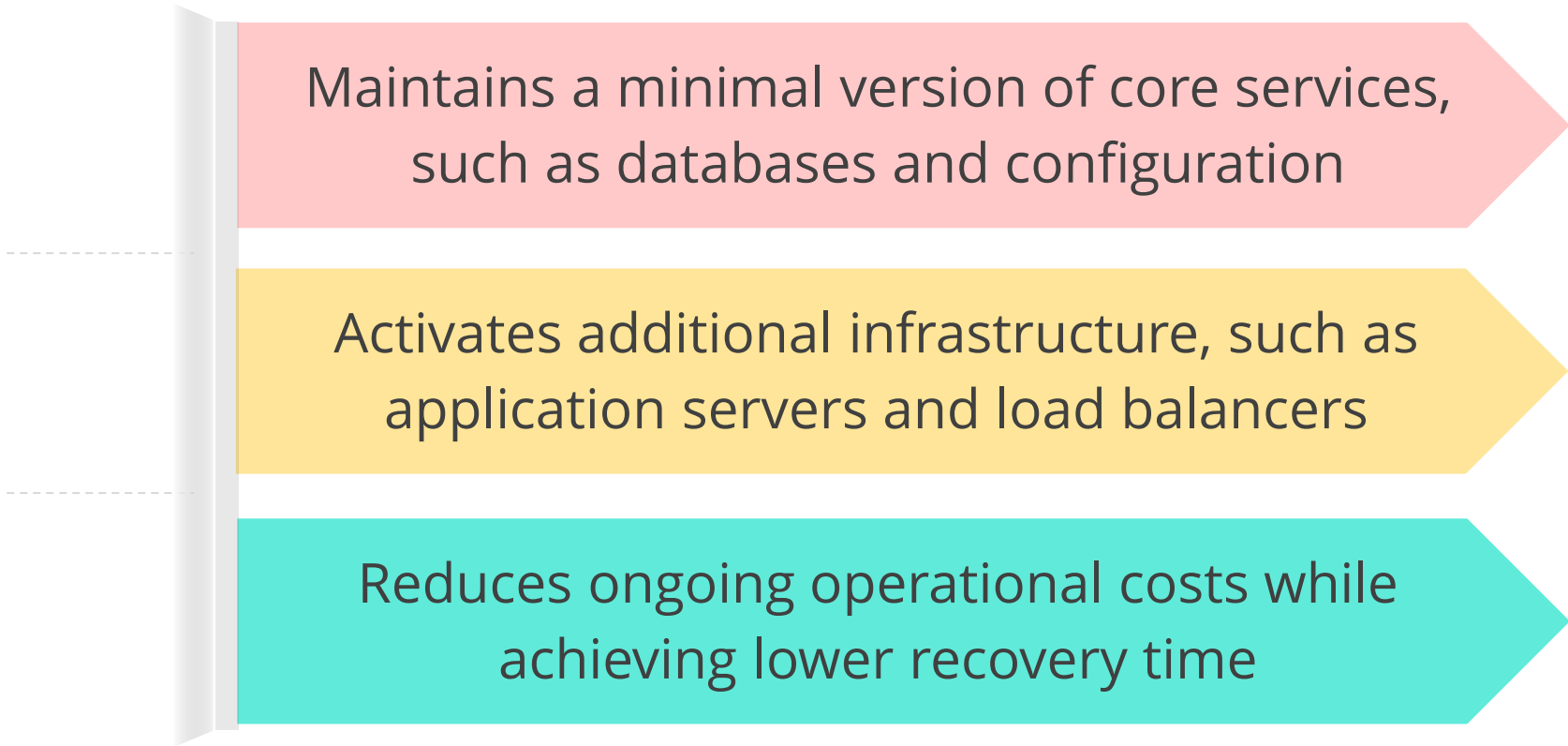
Stores backup regularly in a durable service like S3 or Glacier

Requires manual or scripted recovery steps during a disaster

Has high RTO and RPO due to longer restore times and potential data gaps

Pilot Light Pattern

It keeps essential system components running in standby mode, making it easier to scale up quickly during a disaster.



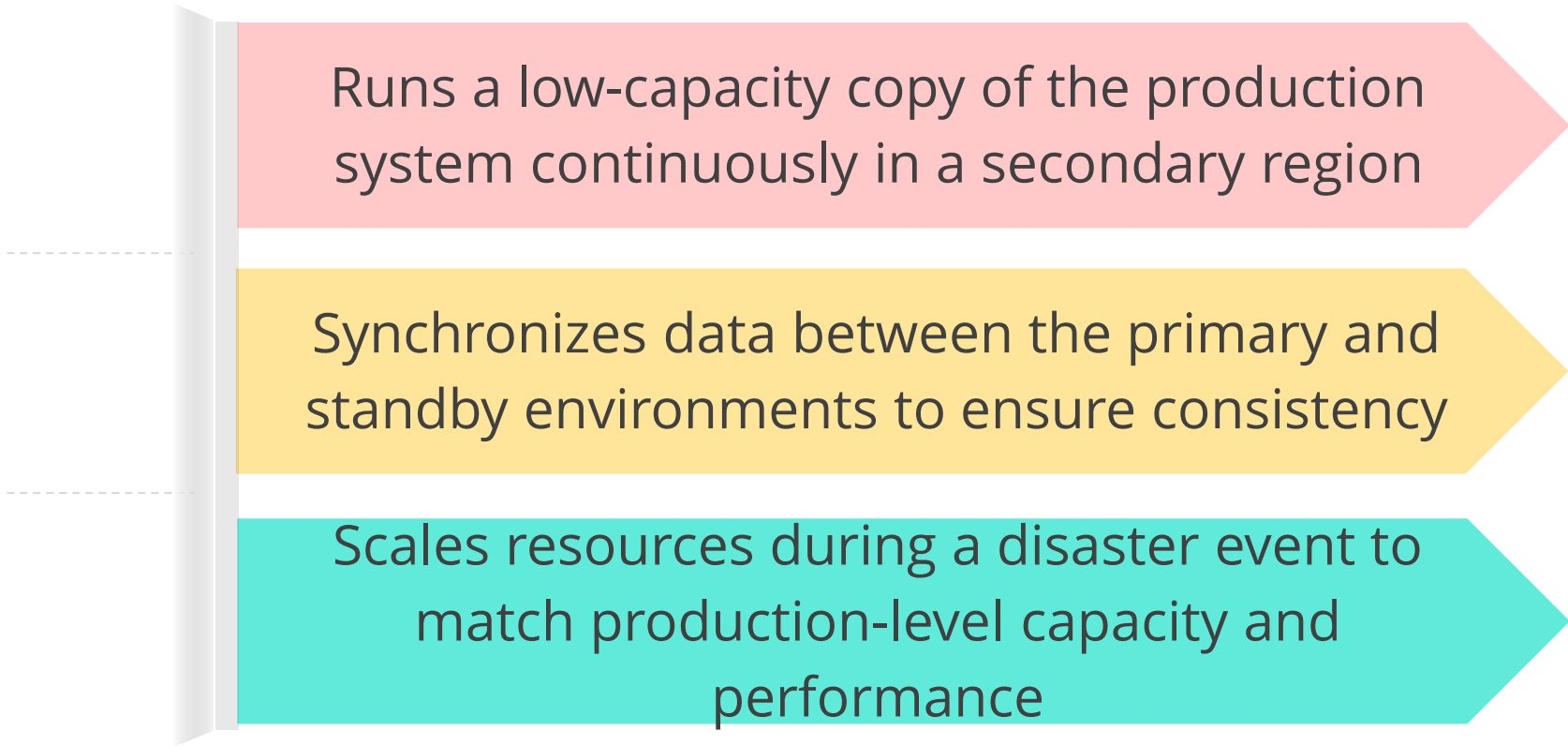
Maintains a minimal version of core services, such as databases and configuration

Activates additional infrastructure, such as application servers and load balancers

Reduces ongoing operational costs while achieving lower recovery time

Warm Standby Pattern

It maintains a partially running version of the production system, ready to scale up during failover.



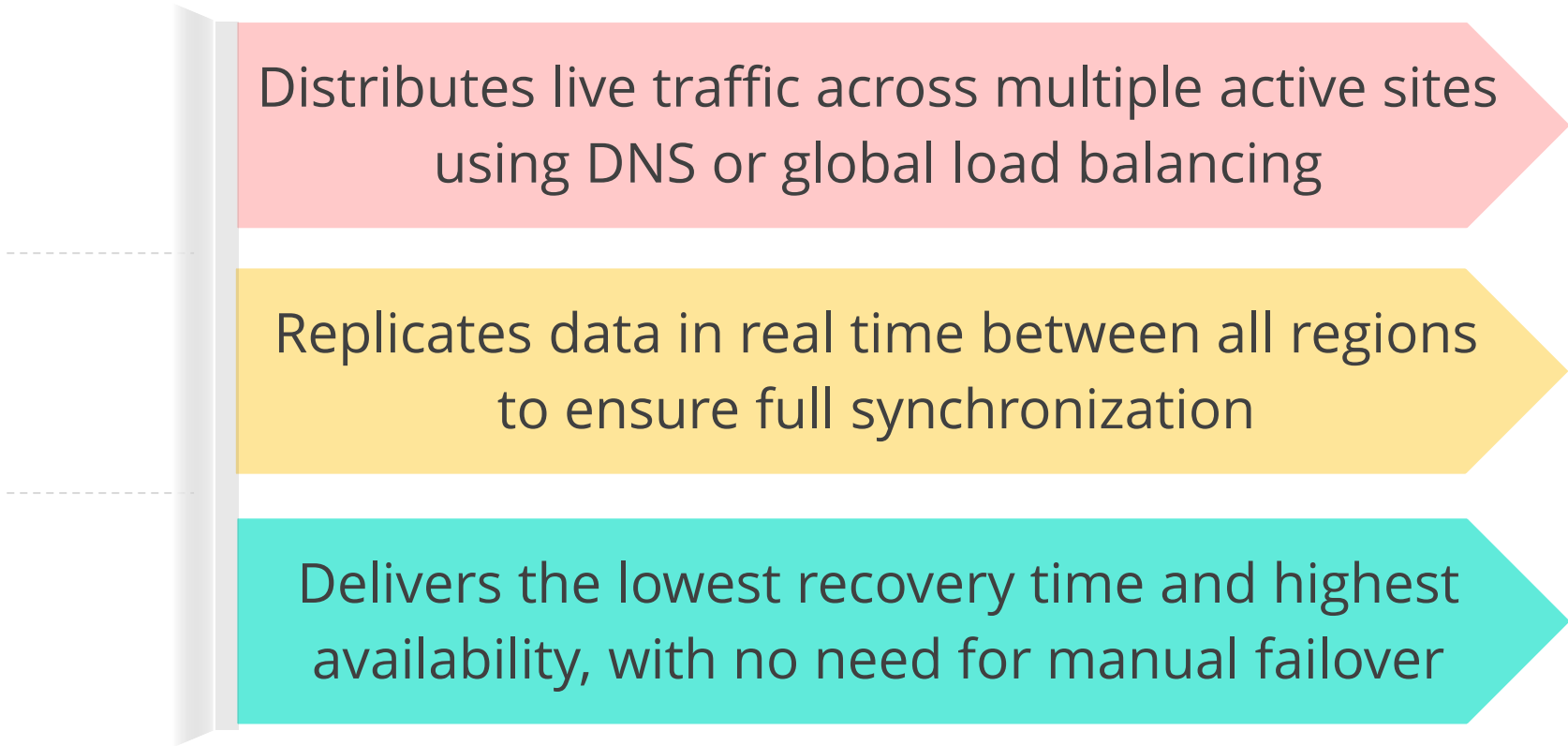
Runs a low-capacity copy of the production system continuously in a secondary region

Synchronizes data between the primary and standby environments to ensure consistency

Scales resources during a disaster event to match production-level capacity and performance

Multi-Site Active

It runs fully functional environments in two or more regions at all times.



Distributes live traffic across multiple active sites using DNS or global load balancing

Replicates data in real time between all regions to ensure full synchronization

Delivers the lowest recovery time and highest availability, with no need for manual failover

AWS Services for DR

It provides several services that help automate failover, distribute traffic, and maintain capacity during disasters. The following are some of the key services:

Amazon Route 53
(DNS Failover)

Elastic Load
Balancer (ELB)



Auto Scaling Groups
(ASG)

AWS Service: Amazon Route 53 (DNS Failover)

It is a scalable domain name system service that uses health checks and routing policies to direct users to healthy endpoints during failures.

- Detects unhealthy endpoints using health checks and routing policies

- Redirects traffic automatically to standby environments across regions

- Supports active-passive and active-active architectures for resilience

AWS Service: Elastic Load Balancer (ELB):

It is a managed service that automatically distributes traffic across healthy targets to improve availability and performance.

- Distributes request across instances, availability zones, or regions

- Removes failed targets from rotation and route traffic to healthy ones

- Improves performance and reliability with intelligent, health-based routing

AWS Service: Auto Scaling Groups

It is a capacity management service that maintains application availability by adding, removing, or replacing instances based on health checks and scaling policies.

Monitors key metric and trigger scale-out or scale-in actions automatically

Replaces failed instance to preserve desired capacity and health

Recovers quickly from a spike or failure by launching instances on demand

Use Case

Large Financial Customer - Aurora Global or Multi-Region Failover



Problem

- The client's customer portal needed sub-minute failover in case of AZ outages and single-digit minute recovery between AWS regions, while minimizing human error in failover.
- They also needed automated health detection, strong consistency and data replication, and minimal downtime during migrations or failures.
- The existing architecture lacked robust DR across regions and relied on manual or slow processes for region failover.

Use Case

Large Financial Customer - Aurora Global or Multi-Region Failover



Solution

- They used Amazon Aurora Global Database (PostgreSQL) to replicate data across regions with low lag and built on RDS Proxy plus automation for failover orchestration.
- Within a region, they used Multi-AZ deployments for HA to get failover at the availability zone level in sub-minute time.
- They also automated failure detection, failover, and recovery logic, and built in resilience to AWS control-plane faults so failover would proceed even if AWS control plane had issues.

Use Case

Large Financial Customer - Aurora Global or Multi-Region Failover



Outcome

- They achieved sub-minute AZ failovers and single-digit minute cross-region recovery, meeting their RTO or RPO SLAs.
- The automatic failover flow reduced human intervention and the risk of operator mistakes in disaster events.
- The system became more resilient: even if a whole AZ or region suffers a failure, failover is faster, safer, and more reliable.

Use Case

US Insurance Company - Multi-Region DR for 3-Tier Apps



Problem

- A US insurance company needed a reliable disaster recovery (DR) solution for its 3-tier applications to ensure business continuity.
- The existing setup could not meet stringent Recovery Time Objective (RTO) and Recovery Point Objective (RPO) requirements during regional or availability zone failures.
- Manual failover processes introduced delays and increased the risk of human error, limiting resilience and exposing the organization to downtime and operational disruptions.

Use Case

US Insurance Company - Multi-Region DR for 3-Tier Apps



Solution

- Implemented a Pilot Light or Warm Standby architecture with continuous cross-region database replication to maintain data availability.
- Automated failover using EventBridge plus Lambda health checks, triggering Jenkins pipelines for controlled recovery actions.
- Used Route 53 with ALB for DNS failover, backed by approval gates to balance automation with operational control.

Use Case

US Insurance Company - Multi-Region DR for 3-Tier Apps



Outcome

- Achieved RTO or RPO under 15 minutes for critical 3-tier applications across regions
- Standardized a repeatable, automated DR process for both Windows and Linux workloads with multiple database engines
- Reduced human error and improved reliability through automation, while still allowing manual oversight for failover decisions

Quick Check



Your e-commerce app on AWS requires in-region DB failover, read scaling, $RTO \leq 30 \text{ min}$ / $RPO \leq 5 \text{ min}$ across regions, data deletion protection, and DNS failover. Which setup fits best?

- A. Single-AZ RDS + Backup and Restore + S3 Lifecycle
- B. Multi-AZ RDS + Read Replicas + Warm Standby + S3 Versioning + Route 53 Failover
- C. Aurora Serverless only + CloudFront
- D. Read Replicas only + Manual DNS switch

Key Takeaways

- Cloud architectures should tolerate faults, maintain high uptime, and scale wisely, using vertical scaling for quick boosts and horizontal scaling for elasticity and isolation.
- Cloud deployments should span boundaries, using Multi-AZ for local outages and Multi-Region for regional failures and faster global performance.
- Data resilience is enhanced by combining RDS Multi-AZ for automatic failover, Read Replicas for read scaling, and backup tools like AWS Backup, EBS Snapshots, and S3 Versioning.
- Disaster recovery planning requires aligning business needs with the right strategy, from Backup and Restore to Pilot Light, Warm Standby, or Multi-site Active.
- Automated recovery mechanisms such as Route 53 DNS failover, Elastic Load Balancers, and Auto Scaling enable rapid failover, self-healing, and continuous application availability.



Deploying a Resilient Web Application with Auto Scaling and Multi-AZ RDS

Duration: 25 Mins.

Project agenda: To design and implement a highly available web application architecture on AWS that can withstand heavy user traffic while ensuring reliability, scalability, and uninterrupted service delivery. The solution will leverage Multi-AZ Amazon RDS deployments for database high availability and fault tolerance, ensuring automatic failover in case of instance or availability zone failure.

Description: You are tasked with implementing a highly available and scalable web application architecture on AWS to ensure reliability, fault tolerance, and uninterrupted service delivery during high user traffic. The solution leverages Multi-AZ RDS for database resilience and automatic failover, Auto Scaling Groups for dynamic adjustment of application instances based on demand, and an Elastic Load Balancer for distributing incoming traffic efficiently. By combining these AWS services with monitoring, logging, and health checks, the architecture will provide a resilient, self-healing environment that can handle unexpected spikes, maintain business continuity, and optimize resource usage.



Deploying a Resilient Web Application with Auto Scaling and Multi-AZ RDS

Duration: 25 Min.

Perform the following:

1. Create a Multi-AZ MySQL instance in the RDS service
2. Create an Auto Scaling mechanism for a Web application
3. Validate Auto Scaling and Multi-AZ mechanism

Expected deliverables: A fully implemented cloud architecture featuring a Multi-AZ RDS database and Auto Scaling-enabled application tier, an operational load-balanced web application accessible under varying traffic loads, a configuration guide detailing setup steps and architectural decisions, monitoring and alerting dashboards with CloudWatch for performance and reliability tracking, documented test scenarios demonstrating high availability and scalability under simulated traffic spikes, and a final report summarizing outcomes, performance results, and recommendations for further optimization.



Q&A

